

Lightweight Wind and Velocity Estimation for GPS-Denied Drone Navigation

Using Physics and Lightweight ML to Maintain Spatial Awareness in Open-Air Environments

Alexander Alfaro

Institute of AI and Machine Learning

Blekinge Institute of Technology

Karlskrona, Sweden

almi21@student.bth.se

I. INTRODUCTION

Maintaining spatial awareness in outdoor and urban environments remains a critical challenge for autonomous Unmanned Aerial Vehicles (UAVs). Current guidance systems depend heavily on Global Navigation Satellite Systems (GNSS), leaving them vulnerable in GPS-denied environments. When the satellite signal is lost, the UAV must fall back to its onboard Inertial Measurement Unit (IMU); During which the flight controller relies on dead reckoning to estimate position from raw IMU data. The IMU measures the total force on its body frame; this means external forces like wind are indistinguishable from its own propulsion, causing error and drift to occur. [1], [2].

The primary objective of this project was to develop 3 lightweight machine learning models to maintain spatial awareness during GPS denial. The approach was based on the fundamental relationship of the wind triangle $\mathbf{V}_G = \mathbf{V}_a + \mathbf{V}_w$, in which the drone's ground velocity $\mathbf{V}_G$ (its motion observed from the Earth frame) equals its air speed $\mathbf{V}_a$ (its velocity through the surrounding air mass) plus the wind velocity $\mathbf{V}_w$ (the motion of the air itself relative to the ground). From this relationship, 3 models were derived: Model 1 predicts ground velocity ($\mathbf{V}_G$) from IMU telemetry as the primary drift corrector (implemented with KNN regression); Model 2 predicts wind velocity ($\mathbf{V}_w$) directly from an expanded feature set that augments IMU telemetry with GPS velocity and Blade Element Momentum (BEM) theory-derived aerodynamic variables (also KNN); and Model 3 was designed to predict airspeed ($\mathbf{V}_a$) using the same classical regression approaches (KNN and polynomial) in order to close the wind triangle. Models 1 and 2 met their objectives, but Model 3 effectively learned nothing and is therefore excluded from the deployed pipeline; the supporting analysis is presented in Section III.

The pipeline concept positioned Model 1 as the initial velocity corrector while Model 2 converges, after which Model 3 would close the wind triangle for a complete state estimate. To ensure a reliable, reproducible, and structured methodology, the project follows the CRISP-DM (CRoss

Industry Standard Process for Data Mining) reference model [3], [4], whose phases are summarized in Figure 1. Flight data was generated systematically using a PX4 Software-in-the-Loop (SITL) simulation pipeline coupled with Gazebo. The following sections detail the execution of the six CRISP-DM phases, from initial business understanding and flight data collection through to model evaluation and SHAP-based explainability analysis.

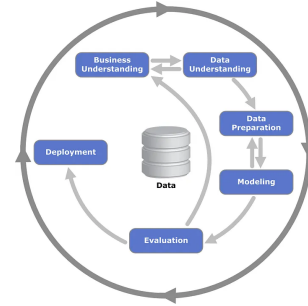


Fig. 1: Phases of the CRISP-DM reference model

II. METHODOLOGY (CONCERNING THE CRISP-DM)

A. Business Understanding

The application domains for this work, including disaster response, urban logistics, dense forest operations, and contested airspace, all of which require sustained position awareness without reliable satellite signals. The business need is a lightweight and cheaper, onboard method for estimating wind velocity and correcting IMU drift when GNSS is unavailable.

Following the CRISP-DM framework [3], [4], we developed 3 models derived from the wind triangle $\mathbf{V}_G = \mathbf{V}_a + \mathbf{V}_w$: Model 1 predicts the IMU velocity-error to get better position estimates; Model 2 estimates wind velocity; Model 3 predicts airspeed to close the triangle. Success was defined as measurable improvement over uncorrected dead reckoning: Model 1 is evaluated during training on MAE (Mean absolute error), and during real use, benchmarked against the median and 10th-to-90th percentile range of position drift after one

minute of GPS-denied flight; Model 2 is evaluated during training with MAE and R^2 . During real use, Model 2 is benchmarked on convergence time for extracting the correct wind vector, and Model 3 on whether machine learning can reduce the intrinsic variance of analytically computed airspeed.

B. Data Understanding

Flight data was generated using a PX4 Software-in-the-Loop (SITL) simulation coupled with Gazebo for visuals. The simulated environment used the baylands_windy world, which is based on Baylands Park, a real geographic location in California, providing terrain and simplified wind dynamics. The wind was configured using the baylands_windy world's SDF (Simulation Description Format) file to configure wind conditions. The x500 quadcopter airframe was used. Two distinct flight patterns were flown, one for training and one for validation. The training pattern was a square-and-lawnmower trajectory tracing a 400 m clockwise square with five north-south strips inside at 30 m altitude, producing long straight legs with sustained constant-heading flight that provide steady-state wind response across all four directions. The validation pattern was a straight-line delivery route at 30-40 m altitude consisting of a 500 m outbound leg followed by a 500 m return leg (1000 m total per flight). Five wind conditions were each flown twice, once along a North-South axis and once along an East-West axis, giving 10 validation flights in total. All flights operated under OFFBOARD control at a 20 Hz setpoint rate via pymavlink.

MAVLink telemetry was logged at 10 Hz through a dedicated telemetry capture script. Each CSV row contains a timestamp, IMU integrated NED velocities (V_n , V_e , V_d)¹, attitude (roll, pitch, yaw), angular rates, linear acceleration, GPS velocity, throttle, flight mode, arming state and Blade Element Momentum (BEM) theory-derived aerodynamic features. The full dataset comprises 53 training flights across 24 wind conditions, with $wind_x$ ranging from -6 to $+10$ m/s and $wind_y$ ranging from -5 to $+10$ m/s. The validation set is comprised of 10 flights across 5 wind conditions. The training set contains 216,276 rows after filtering, and the validation set contains 17,051 rows after filtering, representing approximately 6 hours of simulated flight time.

Model 1 uses 12 features²: IMU velocity (3), attitude (3), angular rates (3), and linear acceleration (3). Model 2 uses 20 features: GPS velocity (3), attitude (3), angular rates (3), linear acceleration (3), gyroscope (3), throttle, climb rate, and BEM-derived features (3). Relative to Model 1, IMU velocity is replaced by GPS velocity, and gyroscope, throttle, climb rate, and BEM features are added. Model 3 uses 15 features: throttle, attitude (3), angular rates (3), linear acceleration (3), gyroscope (3), and BEM-derived features (2). Preprocessing removed rows where the vehicle was disarmed, GPS was unlocked, or values were NaN. Statistical outliers exceeding $|z| > 5$ were also removed.

¹In the dataset they are logged as imu_vel_n, imu_vel_e and imu_vel_d

²Column names by group (IMU velocity: imu_vel_n, imu_vel_e, imu_vel_d; GPS velocity: gps_vx, gps_vy, gps_vz; attitude: roll, pitch, yaw; angular rates: rollspeed, pitchspeed, yawspeed; linear acceleration: imu_xacc, imu_yacc, imu_zacc; gyroscope: imu_xgyro, imu_ygyro, imu_zgyro; throttle: throttle; climb rate: climb_rate; BEM features: bem_total_thrust, bem_expected_accel_x, bem_wind_residual_x NOTE: Model 3 uses only the first two)

climb rate, and BEM-derived features (3). Relative to Model 1, IMU velocity is replaced by GPS velocity, and gyroscope, throttle, climb rate, and BEM features are added. Model 3 uses 15 features: throttle, attitude (3), angular rates (3), linear acceleration (3), gyroscope (3), and BEM-derived features (2). Preprocessing removed rows where the vehicle was disarmed, GPS was unlocked, or values were NaN. Statistical outliers exceeding $|z| > 5$ were also removed.

C. Data Preparation

To compute the velocity features, we integrated the body-frame linear acceleration over time. We started from the first row where the vehicle was armed then at each step with $0 < \Delta t < 0.5$ s the body-frame specific force was rotated into NED using a ZYX Euler matrix, gravity removed ($+9.81$ m/s², down-positive NED)³, and the resulting NED-frame acceleration integrated to yield imu_vel_n/e/d. The Model 1 label was then computed as $imu_vel_err = gps_velocity - imu_velocity$. Raw GPS velocity was scaled from cm/s to m/s by dividing by 100.0.

Aerodynamic features were derived from Blade Element Momentum (BEM) theory using servo PWM commands. Parameters were taken from the PX4 SITL airframe configuration: $K_t = 8.54858 \times 10^{-6}$, maximum rotor angular velocity 1000 rad/s, drone mass 2.0 kg. Per-motor thrust was computed as $T_i = K_t \cdot \omega^2$ after clipping PWM to $[0, 1000]$ and converting to rad/s, then summed across the four motors. Forward force was obtained as $total_thrust \cdot \sin(pitch)$ and expected acceleration as $forward_force/mass$. The wind residual, defined as the difference between expected aerodynamic acceleration and the measured x-axis acceleration, serves as a proxy for external wind disturbance. During dataset generation, the horizontal wind parameters ($wind_x$, $wind_y$) were recorded as the filenames of the respective CSVs rather than stored within the files themselves. These labels were extracted directly from the filenames during processing, with $wind_z$ held at zero. Model 3 labels were then computed kinematically as the difference between GPS ground velocity and the true wind vector.

Row filtering removed disarmed rows, rows with zero GPS velocity (indicating no satellite lock), and rows missing required features or labels. Statistical outliers exceeding $|z| > 5$ on any model feature were excluded. The 53 training and 10 validation flights were simulated separately, preserving flight integrity. Validation flights covered the wind pairs (0, 0), (0, 5), (4, -3), (5, 0), and (8, 0).

³In the NED-frame, we filter out baseline gravity so a resting state equals 0. This is because raw IMU data will record -9.81 due to how the sensor works. Now, because Down is the positive direction in this frame, if the drone accelerates down we get a positive value, and if it accelerates up we get a negative value. This filters out gravity so we only track actual movement

A cardinal balancing step was applied for Model 1 to address directional performance imbalance. Heading angles were bucketed into four quadrants (N/E/S/W) using $\pm 45^\circ$ cones, and three strategies were evaluated via Optuna: no balancing, undersampling to the minority bucket, and oversampling with replacement to the majority bucket. The best configuration used oversampling, distance-weighted KNN, and RobustScaler (centered on the median and scaled by interquartile range), fit on training data only to avoid leakage.

D. Modeling

The original proposal targeted an equation-based gray-box approach following the methodology of Sun *et al.* [5], which proposed a physics-based model on how a drone’s aerodynamics change at high speeds. In the work, they showed that these aerodynamic effects only appear in real flights and not in simulated runs. Since our project runs on data simulated in (PX4/Gazebo), where the rotor thrust is modeled as $T = K_t \omega^2$, this cannot describe the effects that are present in higher-speed drone flight. This meant that features built to capture these effects had nothing to describe in the simulator and only added error to the model. The final pipeline instead keeps physics-informed features that match what the simulator actually computes and pairs them with a data-driven KNN regressor. The result is gray-box at the feature level rather than the equation level, while still respecting the course constraint to avoid deep-learning solutions.

Model 1 was tuned over these balancing strategies (none, undersample, oversample), the scaler (StandardScaler, RobustScaler), the neighbor count k (a log-uniform integer between 3 and 3000), and the weighting scheme (uniform, distance). Two independent search strategies were run on identical seeds: a random sampler and a TPE-driven Optuna sampler at 200 trials. Both converged on hyperparameter regions producing essentially identical validation performance (mean global MSE difference 0.08%), indicating that KNN with the current feature set has saturated on this dataset. The reported Model 1 configuration uses oversampling, RobustScaler, $k = 30$, and distance-weighted KNN.

The choice of KNN with attitude and IMU-acceleration features for the wind estimator follows Crowe *et al.* [6], who established KNN over drone attitude and accelerometer channels as a viable supervised baseline for windspeed estimation on a hovering quadcopter. Hyperparameter selection for Model 2 used a grid search over four model families (ridge regression, polynomial regression of degree 2 and 3, and KNN) with the best configuration chosen by validation MAE. For KNN, the neighbour count k was swept over the set $\{1, 2, \dots, 10, 12, 15, 20, 25, 30, 40, 50, 75, 100, 150, 200\}$. The selected Model 2 configuration is a KNN regressor

with $k = 15$, StandardScaler, and distance-weighted neighbors. Model 3 was deliberately not subjected to a full hyperparameter sweep because its classical configurations produced results indistinguishable from omitting the model entirely (see Section III); further tuning could not have rescued an approach that was already no better than omitting the model itself.

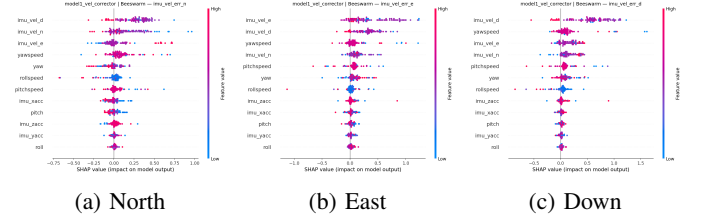


Fig. 2: Model 1 SHAP beeswarm summaries per predicted velocity-error axis. Each row is one input feature; horizontal position shows the SHAP value (impact on the prediction); color encodes the feature value.

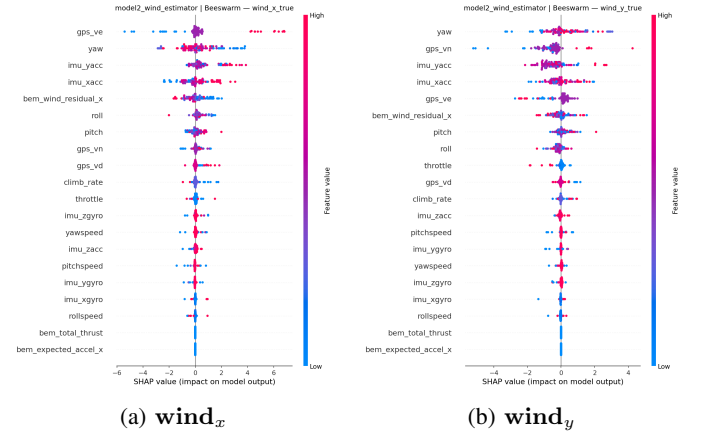


Fig. 3: Model 2 SHAP beeswarm summaries per predicted wind-vector component.

1) *Explainable AI (XAI) Considerations:* SHAP was used to analyze Model 1 predictions per output axis (Figure 2) and Model 2 predictions per wind component (Figure 3), giving both feature ranking and the direction of each effect.

For the imu_vel_err_n target, imu_vel_d dominates: forward flight requires a forward body tilt that couples a downward velocity component to the thrust vector. The own-axis imu_vel_n ranks second and the cross-axis imu_vel_e third, reflecting lateral tilt used to hold heading against side wind. The imu_vel_err_e target mirrors this pattern with imu_vel_e leading and imu_vel_d second; for imu_vel_err_d , imu_vel_d remains the dominant predictive feature, followed notably by yawspeed and horizontal velocity errors.

Across all three targets, angular rates generally hold high importance, with yawspeed consistently ranking in the top four, though the static yaw angle consistently outranks

rollspeed (and pitchspeed for imu_vel_err_n). The raw accelerometers rank in the lower half, but notably outrank the static pitch and roll angles. The lower importance of raw acceleration suggests that the velocity features already largely encode this information. This indicates that the principal lever in this pipeline is feature formulation rather than feature availability.

For Model 2 (Figure 3), the wind_x prediction is driven by gps_ve (the east-axis ground velocity that aligns with wind_x), yaw , imu_yacc , imu_xacc , and $\text{bem_wind_residual}_x$, the last of which is the gap between the acceleration predicted from motor thrust and the measured imu_xacc , and therefore captures the aerodynamic force on the whole airframe. The wind_y attributions are nearly the same set, with yaw narrowly ahead of gps_vn and the accelerometer pair contributing next; the absence of a symmetric $\text{bem_wind_residual}_y$ feature is a known asymmetry in the feature pipeline and is revisited in the Limitations section.

E. Evaluation

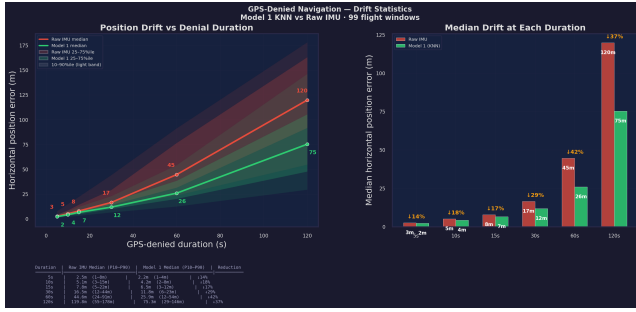


Fig. 4: Model 1 horizontal position drift on validation flights: median, p10, and p90 of integrated position error versus elapsed time, for the raw IMU baseline (red) and the M1-corrected stream (green), across 99 flight windows; each flight cuts off GPS at a random time within the 10 validation flights and is compared to dead reckoning.

All evaluations were performed on the validation flights, which were never seen during training, hyperparameter search, or calibration. Three deployment-relevant evaluations were made. For Model 1, the metric of interest is measuring how far the position drifts over time: the corrected NED velocity was integrated forward from a known starting position and the Euclidean horizontal drift was recorded at 5, 10, 15, 30, 60, and 120 second windows, against the SITL ground-truth trajectory (Figure 4). We compared dead-reckoning drift with and without Model 1 velocity correction. For Model 2, we measured how long the mode took to converge within ± 1 m/s of the true value for at least 5 s. This evaluation was done per output axis and as a joint condition (both axes simultaneously inside the band). The same procedure was repeated under simulated GPS denial as described in Section III-B to see how much worse it gets during deployment. For Model 3, per-axis

MAE, RMSE, and R^2 were reported against the analytic floor described in Section III-C. A per-flight heading check (north / east / south / west cones, $\pm 45^\circ$) was additionally generated to verify that errors were not concentrated in a single heading regime, but is omitted from this report for space reasons.

F. Deployment

The deployed pipeline consists of two KNN models, Model 1 for velocity correction and Model 2 for wind estimation. Both models start running when the drone starts; since a single Model 2 prediction is noisy, we instead wait for a more stable estimate based on the mode, which is the peak of the distribution of Model 2's predictions. Model 1's job is to keep location estimates accurate by predicting the IMU accumulated error. With this setup, we can still make Model 2 work even in GPS-denied environments because Model 1 provides cleaner IMU velocity estimates. All validations were performed on PX4 SITL telemetry data; physical tests were left for future work and are discussed in Section V.

III. RESULTS AND ANALYSIS

A. Model 1: Velocity Corrector

Model 1 corrects accumulated velocity error to keep position estimates usable during GPS denial, making it the backbone of the pipeline. We recorded the Euclidean horizontal drift at 5, 10, 15, 30, 60, and 120 second windows. We computed this by selecting a random GPS-cutoff start point for each window across all 10 validation flights. (Figure 4) shows that the differences were minor in the 5-30 second time windows, but as the error grows, the distance between the corrected version and raw version becomes more noticeable. The reason we chose the 60-second mark to benchmark is that, at longer durations, the drift becomes too high for real use. The 60-second mark hits the sweet spot; the drone has significant drift, but in open terrain a 26-meter drift is still recoverable. At 60 seconds, we measured an average drift of 26 m, compared to 45 m with standard dead reckoning. The spread improved as well; at the 10th percentile, drift went from 24 m dead reckoning to 12 m, and at the 90th percentile from 91 m to 54 m.

B. Model 2: Wind Estimator

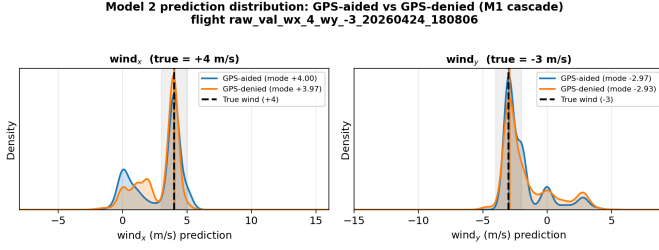


Fig. 5: Model 2 prediction distributions comparing the GPS-aided baseline against a simulated GPS-denied cascade (using Model 1 corrected velocity). The distribution peak (mode) remains tightly locked on the true wind vector despite cascading errors.

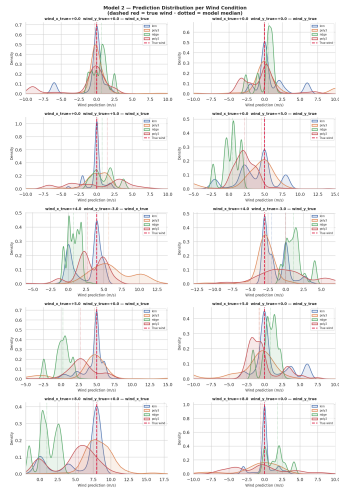


Fig. 6: Comparison of Model 2's wind prediction distributions across four regression models (KNN, Ridge, Poly2, and Poly3) evaluated under various true wind conditions. The selected KNN model consistently shows its distribution peak (mode) closely aligns with the true wind value (dashed red line). The mode estimator was selected over the mean because the predictive distributions are frequently asymmetric, making the peak a more reliable and robust point estimate.

Model 2 was implemented as a KNN regressor over a 20-feature set that combines IMU telemetry with GPS velocity, gyroscope channels, throttle, climb rate, and three BEM-derived aerodynamic variables. Rather than report a per-timestep prediction, predictions were aggregated into a distribution over a sliding time window and the wind-speed estimate was taken as the *mode* (peak) of that distribution. This choice was motivated by the observation that the distribution peak aligned with the ground-truth wind value more reliably than the windowed mean, as shown in Figure 6. To the best of our knowledge, this strategy and the associated convergence-to-mode metric are not used in the comparable wind-estimation literature reviewed [1], [2], [5], [7], where wind estimates are reported as deterministic point estimates

or filter-posterior means.

With GPS available the model converged to reliable wind estimates on every training flight, with joint median convergence times ranging from 0.5 to 37 seconds; the majority of training flights converged within 13 seconds. On the 10 validation flights, the mode estimator converged on all flights, with a median convergence time of approximately 61 seconds and a 95th percentile of 105 seconds. This indicates that approximately one minute of GPS-available flight data are typically required before wind estimates become reliable enough to support downstream use.

To test operation under GPS denial, a cascade experiment was run in which Model 2 was evaluated on the same 10 validation flights with the $\text{gps_v}\{n, e, d\}$ inputs replaced by Model 1's corrected IMU velocity ($\mathbf{V}_{\text{IMU}} + \hat{\text{err}}$). The cascade locked the true wind in 9 of the 10 flights, and on those nine the converged wind-vector accuracy was indistinguishable from the GPS-aided baseline shown in Figure 5 (wind_x MAE 0.42 m/s, wind_y MAE 0.62 m/s, versus 0.42 and 0.61 m/s with native GPS). The cost was time-to-lock: the joint mode-convergence median rose from 61 s to 82 s, while the 95th percentile increased only marginally (from 105 s to 110 s). Since the primary requirement is to converge on the true wind rather than minimize per-sample residuals, this result supports a deployment pattern in which the drone uses GPS to establish a wind lock and can subsequently maintain that estimate through GPS denial during the same flight, at a modest convergence-time penalty.

C. Model 3: Airspeed Predictor (Negative Result)

V_{air} Predictor Error Floor — Validation Set (10 flights)

Estimator	Mean [err] (m/s)	P95 [err] (m/s)	Improvement vs baseline	Error reduction
Predict zero (uncorrected drift)	7.51	15.04	1.0x	0%
Per-flight constant mean	4.48	12.02	1.7x	40%
Low-pass filter (achievable floor)	0.21	1.80	36.1x	97%

Fig. 7: Analytic error floor for Model 3 (V_a) predictions evaluated across the validation set. The low-pass filter establishes the maximum achievable variance reduction (97%), serving as the benchmark that the classical regression models failed to approach.

To test the achievable performance for V_a predictions, we used three simple estimators to find the lowest possible error on the validation set shown in Figure 7. The baseline “predict zero” estimator, representing uncorrected drift, achieved a mean error of 7.51 m/s; a per-flight constant mean predictor reduced this to 4.48 m/s and a low-pass filter on the V_a achieved 0.21 m/s, setting the error floor (a 36-fold improvement over the baseline and a 97% reduction in error). Model 3 was trained with the same classical regression approaches as Model 1 and Model 2 (KNN and polynomial) on a 15-feature subset, but produced results indistinguishable

Estimator / Model	Mean err (m/s)	P95 (m/s)	Reduction vs baseline	Error std (m/s)
Predict zero (uncorrected)	7.51	15.04	0%	4.46
Per-flight mean (oracle)	4.48	12.02	40%	5.68
Ridge (poly ²)	7.91	18.89	-5%	4.61
KNN (K=35)	4.46	19.28	41%	5.79
Shallow MLP (probe)	3.75	17.47	50%	4.80
Low-pass filter (FLOOR)	0.21	1.80	97%	0.62

Fig. 8: Comparison of trained Model 3 airspeed predictors against baseline estimators and the analytic error floor. The best-performing classical model (KNN) achieved a mean error reduction nearly identical to the trivial per-flight mean, demonstrating that it offered no substantial predictive gain over a basic statistical average.

from omitting Model 3 altogether as shown in Figure 8 and is therefore excluded from the deployed pipeline. A shallow neural network was briefly tested and did reduce the error and variance, but was not pursued under the course constraint to avoid deep-learning methods. The negative result is itself instructive: it shows that the IMU relationships required to minimize V_a variance cannot be learned from the available features using classical regression.

IV. DISCUSSION

The three results we got illustrate why some models worked, and others didn’t for IMU-derived state estimation. Model 1 succeeded because the relationship between integrated IMU velocities and yawspeed correlated somewhat well with the IMU velocity error. As confirmed by the SHAP analysis (subsection II-D1), the model relies on predictable flight mechanics, such as a forward body tilt producing a downward velocity component. This is probably why `imu_vel_d` ranks among the highest for the NED velocity error predictions. Also, the integrated IMU velocity features outranked the raw accelerometer channels, showing that the engineered signals, outrank raw sensor measurement.

Model 2 succeeded by looking at the distribution of predictions rather than trusting any single point-wise prediction; individual wind predictions were noisy, but their distribution peaks (mode) managed to reliably find the true wind value within an acceptable time. The GPS-denied cascade experiment confirms that replacing GPS with Model 1’s corrected velocity output keeps the same final accuracy, at only a modest cost to convergence time.

Model 3 failed because the airspeed signal depends on complex aerodynamic effects that classical regression cannot capture. The error floor analysis makes this concrete: a simple per-flight mean estimator reached 40% error reduction, yet the KNN model could not meaningfully beat this baseline despite having 15 features and over 200,000 training samples. Because the low-pass bound shows the airspeed signal is

recoverable in principle, KNN’s failure comes from its inability to untangle these complex aerodynamic effects. However, a brief test with a shallow neural network (MLP) lowered the error beyond the baseline, achieving a 50% reduction. While this network still fell short of the error floor, its improvement over the classical methods suggests the necessary aerodynamic relationships do exist in the feature set, but extracting them likely needs a model that can learn its own representation, which classical methods like KNN cannot.

V. LIMITATIONS

The reported results, including the mode strategy described above, were validated entirely in the PX4 SITL + Gazebo environment on a single x500 airframe in a single world (`baylands_windy`). Real-flight validation on physical hardware is left for future work and is expected to be the primary test of whether the Model 2 behavior seen here transfers to noisier, fully aerodynamic real-rotor data. The training data was slightly imbalanced $wind_x \in [-6, 10]$ m/s but $wind_y \in [-5, 10]$ m/s, so the model has no exposure to the $wind_y = -6$ m/s. In practice this probably only has a small effect on the overall performance of the model, but we did observe that the y axis converged a little slower than x axis in some instances in Model 2. Lastly Model 2 feature set includes a BEM acceleration residual on the x axis but not on the y axis. This was an oversight rather than a deliberate choice, the x residual was adapted from a supporting paper, and a matching y residual was not added. A symmetric y-axis residual should be included in a future version.

VI. CONCLUSIONS

This work applied a CRISP-DM and gray-box methodology to a quadcopter wind-vector estimation, pairing physics-based features (IMU integration, BEM thrust geometry) with classical regressor. A KNN velocity corrector (Model 1) reduces position drift compared to standard dead reckoning. A KNN wind estimator (Model 2), taking the mode of predictions over a moving window, converges to the true wind on all validation flights in under two minutes. The GPS-denied cascade experiment shows that replacing GPS with Model 1’s corrected velocity keeps the same final accuracy at a modest cost to convergence time, supporting a pattern of GPS-aided initial lock followed by GPS-denied maintenance.

The Model 3 negative result shows the airspeed component cannot be learned by classical regression with these features. However, a brief MLP test beat the statistical baselines, suggesting deep learning is likely needed to extract these complex aerodynamic relationships. Future work includes real-flight testing, adding a matching BEM y-axis residual, evaluating a true GPS-denied cascade, and replacing the airspeed component with a deep learning model.

REFERENCES

- [1] E. Kugelberg and O. Andersson, "Wind vector estimation by drone," Master's thesis, KTH Royal Institute of Technology, Stockholm, Sweden, Aug. 2020. [Online]. Available: <https://www.diva-portal.org/smash/get/diva2:1464433/FULLTEXT01.pdf>
- [2] V. J.-P. A. Charmot, "Design of vertical wind estimation algorithms for drones," Master's thesis, KTH Royal Institute of Technology, Stockholm, Sweden, Dec. 2024. [Online]. Available: <https://www.diva-portal.org/smash/get/diva2:1947118/FULLTEXT02.pdf>
- [3] P. Chapman, J. Clinton, R. Kerber, T. Khabaza, T. Reinartz, C. Shearer, and R. Wirth, "CRISP-DM 1.0: Step-by-step data mining guide," The CRISP-DM consortium, Tech. Rep. CRISPWP-0800, Aug. 2000. [Online]. Available: <https://www.kde.cs.uni-kassel.de/lehre/ws2012-13/kdd/files/CRISPWP-0800.pdf>
- [4] R. Wirth and J. Hipp, "Crisp-dm: Towards a standard process model for data mining," *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, pp. 29–39, 01 2000. [Online]. Available: https://www.researchgate.net/publication/239585378_CRISP-DM_Towards_a_standard_process_model_for_data_mining
- [5] S. Sun, C. C. de Visser, and Q. Chu, "Quadrotor gray-box model identification from high-speed flight data," *Journal of Aircraft*, vol. 56, no. 2, pp. 645–661, 2019. [Online]. Available: <https://doi.org/10.2514/1.C035135>
- [6] D. Crowe, R. Pamula, H. Y. Cheung, and S. F. J. De Wekker, "Two supervised machine learning approaches for wind velocity estimation using multi-rotor copter attitude measurements," *Sensors*, vol. 20, no. 19, 2020. [Online]. Available: <https://www.mdpi.com/1424-8220/20/19/5638>
- [7] R. S. Shankar, V. Siotia, and R. O. Panicker, "Drone flight dataset and lightweight LSTM-based wind estimation for semi-autonomous quadcopter control," *IEEE Access*, 2025. [Online]. Available: <https://doi.org/10.1109/ACCESS.2025.3635660>